



DOMAIN

Cloud-Native GenAI Architecture

TEAM

Team of 6

LIVE DEPLOYMENT

klyvix-177846015206.europe-west1.run.app

API BASE

/api/chat/{persona_id}

1 System Architecture Overview

Klyvix uses a modular, cloud-native architecture optimized for real-time AI inference, low latency, and zero single-point-of-failure risk. The application is containerized with Docker and hosted on Google Cloud Run:



2 Core Backend Subsystems

⚡ Streaming Router (`/api/chat`)

FastAPI Async

Accepts prompt, persona ID, and session state. Dispatches request to the appropriate LLM pipeline and yields Server-Sent Events (SSE) directly to the client connection.

🛡️ Multi-LLM Failover Engine

High Availability

Catches Google Gemini HTTP 429 quota exhaustion, 5xx outages, or timeouts and automatically redirects the stream to Groq Llama 3.3 70B within 300ms.

📄 Qdrant Dense Vector RAG

Semantic Search

PDF text is chunked into 400-word segments with 50-word overlap, converted into 768-dimensional embeddings via Gemini, and indexed in Qdrant Cloud for cosine similarity retrieval.

🔒 Auth, Caching & Rate Limiting

Security

Stateless JWT session tokens, bcrypt password hashing, Upstash Redis token-bucket rate limiting (60 req/min), and Neon PostgreSQL persistence.

3 Detailed Request & Data Flow Lifecycles

3.1 Real-Time Server-Sent Events (SSE) Streaming Flow

Stage	Component	Operation & Protocol
1. Dispatch	Browser Client → FastAPI	Client POSTs JSON payload to /api/chat/{persona_id} with Authorization JWT header.
2. Validate	Rate Limiter & Auth Guard	Upstash Redis verifies quota; Pydantic validates message structure and persona configuration.
3. Route	Multi-LLM Router	Binds persona system prompt, sets temperature, attaches retrieved vector context (if applicable).
4. Stream	LLM → StreamingResponse → Client	Tokens are formatted as SSE (data: {"token": "..."}\n\n) and rendered live in the DOM via Marked.js & KaTeX.
5. Complete	Stream Closure	FastAPI emits data: [DONE]\n\n and persists conversation metadata in Neon PostgreSQL.

3.2 Qdrant Vector Retrieval (RAG) Data Pipeline

Step 1: Document Ingestion

User uploads PDF via /api/upload → pypdf extracts text → Regex cleans whitespace and formats metadata.

Step 2: Semantic Chunking

Text is split into 400-word semantic chunks with 50-word sliding overlap to preserve contextual boundaries.

Step 3: Dense Vector Embeddings

Gemini text-embedding-004 generates 768-dimensional vector representations for every chunk.

Step 4: Qdrant Cosine Search

Query vectors are compared against indexed document vectors using cosine similarity; top 3-5 chunks are injected as strict context.

4 Cloud Deployment & Infrastructure Blueprint

Infrastructure Layer	Provider / Resource	Configuration & SLA
Container Compute	Google Cloud Run (Serverless)	Region: europe-west1, Concurrency: 80, CPU: 1.0, Memory: 512MiB, Autoscaling: 0-10.
Vector Database	Qdrant Cloud (Managed Cluster)	Cosine metric, 768 dimensions, HNSW indexing for sub-10ms similarity search.
Relational Database	Neon Serverless PostgreSQL	Connection pooling, auto-suspend during idle, TLS encrypted connection strings.
Distributed Cache	Upstash Serverless Redis	Sub-millisecond token-bucket rate limiting and temporary session cache.